

LEGv8 Instruction Set Simulator

legiss

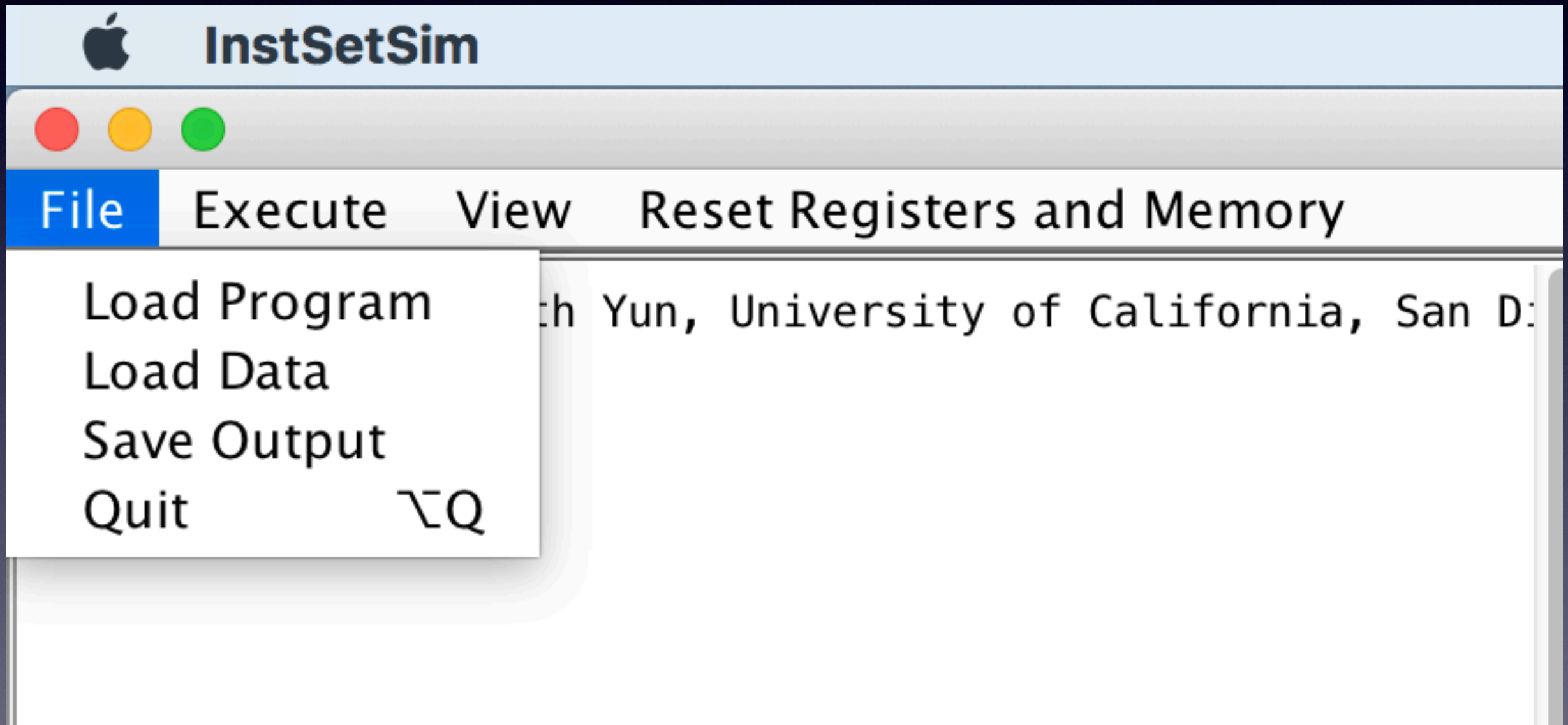
Java ARchive (JAR)

- The instruction set simulator is written in Java
- App name : `legiss.jar`
- Runs on Windows, Linux, and MacOS - just need Java Runtime Environment (JRE)
- 3 window frames: console, registers/data memory, program memory

legiss.jar



Load Program



fact.s

```
    ADDI    X0, XZR, #10
    BL      fact
    STOP

fact:
    SUBI     SP, SP, #32    // allocate stack frame
    STUR     FP, [SP, #0]  // save old frame pointer
    ADDI     FP, SP, #24   // set new frame pointer
    STUR     LR, [FP, #-16] // save return address
    STUR     X0, [FP, #0]  // save argument n

    SUBIS    XZR, X0, #1    // test for n <= 1
    B.LE     L1            // if n <= 1, go to L1
    SUBI     X0, X0, #1
    BL       fact          // if n > 1, call fact(n-1)

    LDUR     X0, [FP, #0]   // return from BL: restore argument n
    MUL      X1, X0, X1     // return n * fact (n ? 1)
    B        done

L1: ADDI     X1, XZR, #1    // return 1

done:
    LDUR     LR, [FP, #-16] // restore return address
    LDUR     FP, [FP, #-24] // restore old frame pointer
    ADDI     SP, SP, #32   // deallocate stack frame

    BR       LR            // return to caller
```

legiss-specific instructions

- `stop` - stops execution
- `putchar` - prints an ASCII character on console
- `putint` - prints a long integer on console

Console

```
LEGv8 Instruction Set Simulator (V0.35 - Updated March 18, 2019)
File  Execute  View  Reset Registers and Memory
Copyright (c) Kenneth Yun, University of California, San Diego. All rights reserved.

> Load Program fact2.s

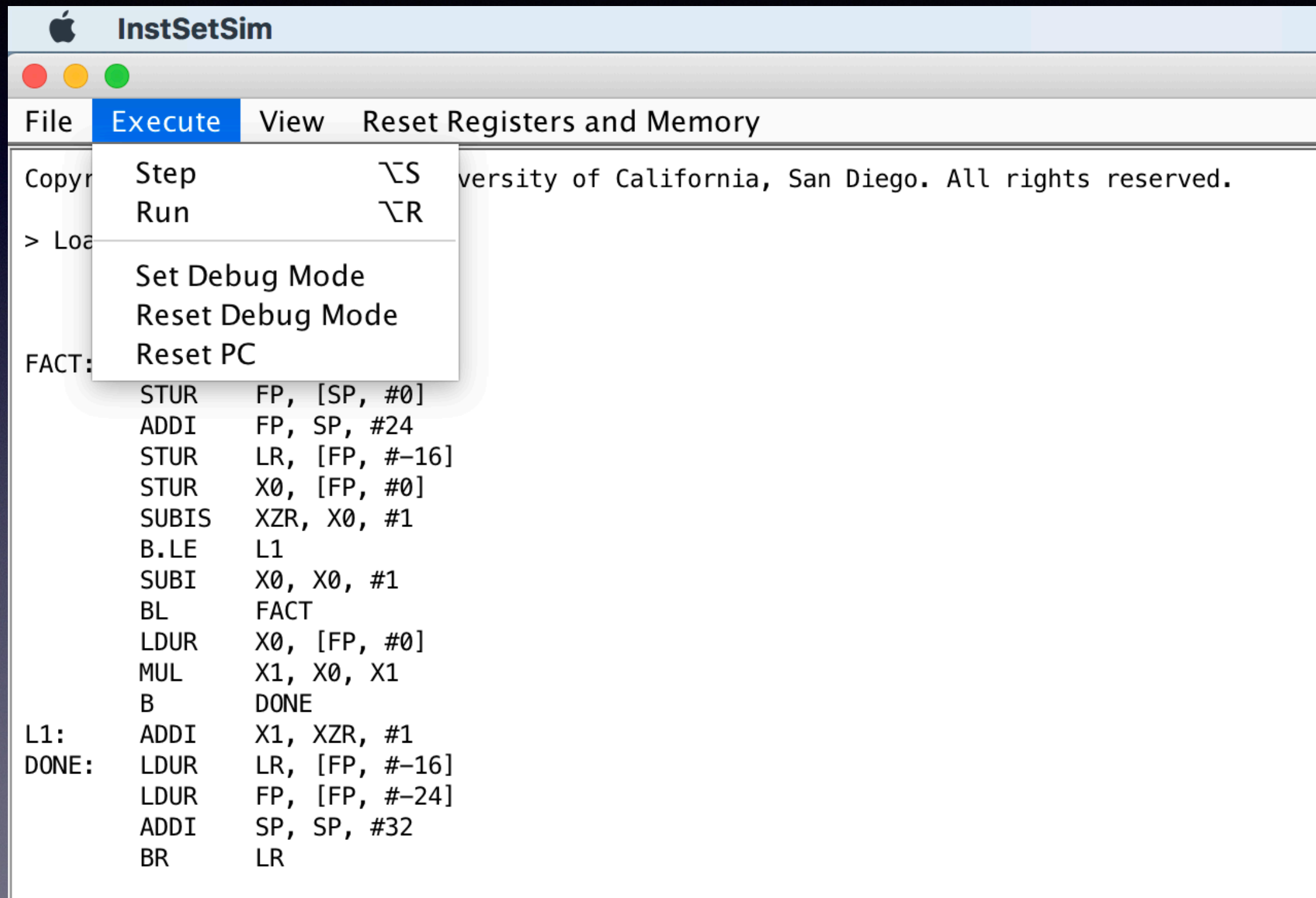
      ADDI    X0, XZR, #10
      BL      FACT
      STOP
FACT:  SUBI    SP, SP, #32
      STUR    FP, [SP, #0]
      ADDI    FP, SP, #24
      STUR    LR, [FP, #-16]
      STUR    X0, [FP, #0]
      SUBIS   XZR, X0, #1
      B.LE    L1
      SUBI    X0, X0, #1
      BL      FACT
      LDUR    X0, [FP, #0]
      MUL     X1, X0, X1
      B       DONE
L1:    ADDI    X1, XZR, #1
DONE:  LDUR    LR, [FP, #-16]
      LDUR    FP, [FP, #-24]
      ADDI    SP, SP, #32
      BR      LR

> Display Program Memory

F [000]          580    0 0 4    ADDI [Rd] 0 [Rn] 31 [Rm] 0 [offset]          a [shamt] 0 [target]          -1
F [001]           37    0 0 1      BL [Rd] 0 [Rn] 0 [Rm] 0 [offset]          0 [shamt] 0 [target]          FACT 3
F [002]           0    0 0 0    STOP [Rd] 0 [Rn] 0 [Rm] 0 [offset]          0 [shamt] 0 [target]          -1
F [003]          FACT 836    0 0 4    SUBI [Rd] 28 [Rn] 28 [Rm] 0 [offset]          20 [shamt] 0 [target]          -1
F [004]          1984    0 0 3    STUR [Rd] 29 [Rn] 28 [Rm] 0 [offset]          0 [shamt] 0 [target]          -1
F [005]          580    0 0 4    ADDI [Rd] 29 [Rn] 28 [Rm] 0 [offset]          18 [shamt] 0 [target]          -1
F [006]          1984    0 0 3    STUR [Rd] 30 [Rn] 29 [Rm] 0 [offset] ffffffff0 [shamt] 0 [target]          -1
F [007]          1984    0 0 3    STUR [Rd] 0 [Rn] 29 [Rm] 0 [offset]          0 [shamt] 0 [target]          -1
F [008]          964    0 0 4    SUBIS [Rd] 31 [Rn] 0 [Rm] 0 [offset]          1 [shamt] 0 [target]          -1
F [009]           84    0 13 1    B.LE [Rd] 0 [Rn] 0 [Rm] 0 [offset]          0 [shamt] 0 [target]          L1 15
F [010]          836    0 0 4    SUBI [Rd] 0 [Rn] 0 [Rm] 0 [offset]          1 [shamt] 0 [target]          -1
F [011]           37    0 0 1      BL [Rd] 0 [Rn] 0 [Rm] 0 [offset]          0 [shamt] 0 [target]          FACT 3
F [012]          1986    0 0 3    LDUR [Rd] 0 [Rn] 29 [Rm] 0 [offset]          0 [shamt] 0 [target]          -1
F [013]          1240   31 0 5    MUL [Rd] 1 [Rn] 0 [Rm] 1 [offset]          0 [shamt] 0 [target]          -1
F [014]           5    0 0 1      B [Rd] 0 [Rn] 0 [Rm] 0 [offset]          0 [shamt] 0 [target]          DONE 16
F [015]          L1 580    0 0 4    ADDI [Rd] 1 [Rn] 31 [Rm] 0 [offset]          1 [shamt] 0 [target]          -1
F [016]          DONE 1986    0 0 3    LDUR [Rd] 30 [Rn] 29 [Rm] 0 [offset] ffffffff0 [shamt] 0 [target]          -1
F [017]          1986    0 0 3    LDUR [Rd] 29 [Rn] 29 [Rm] 0 [offset] ffffffff8 [shamt] 0 [target]          -1
F [018]          580    0 0 4    ADDI [Rd] 28 [Rn] 28 [Rm] 0 [offset]          20 [shamt] 0 [target]          -1
F [019]          1712    0 0 8      BR [Rd] 30 [Rn] 0 [Rm] 0 [offset]          0 [shamt] 0 [target]          -1

>
```

Step/Run



Program Memory

> Step

F [000] 580 0 4 ADDI [Rd] 0 [Rn] 31 [Rm] 0 [offset] a [shamt] 0 [target] -1

>

Program									
BP	Line	Label	Opcode	Rd/Rt	Rn	Rm	Imm/Offset	Shamt	Target
☐	0		ADDI	0	31	0	10	0	
☒	1		BL	0	0	0	0	0	FACT
☐	2		STOP	0	0	0	0	0	
☐	3	FACT	SUBI	28	28	0	32	0	
☐	4		STUR	29	28	0	0	0	
☐	5		ADDI	29	28	0	24	0	
☐	6		STUR	30	29	0	-16	0	
☐	7		STUR	0	29	0	0	0	
☐	8		SUBIS	31	0	0	1	0	
☐	9		B.LE	0	0	0	0	0	L1
☐	10		SUBI	0	0	0	1	0	
☐	11		BL	0	0	0	0	0	FACT
☐	12		LDUR	0	29	0	0	0	
☐	13		MUL	1	0	1	0	0	
☐	14		B	0	0	0	0	0	DONE
☐	15	L1	ADDI	1	31	0	1	0	
☐	16	DONE	LDUR	30	29	0	-16	0	
☐	17		LDUR	29	29	0	-24	0	
☐	18		ADDI	28	28	0	32	0	
☐	19		BR	30	0	0	0	0	

Set Breakpoint and Run

> Set Breakpoint @ Line 11

> Run

>

Program

BP	Line	Label	Opcode	Rd/Rt	Rn	Rm	Imm/Offset	Shamt	Target
☐	0		ADDI	0	31	0	10	0	
☐	1		BL	0	0	0	0	0	FACT
☐	2		STOP	0	0	0	0	0	
☐	3	FACT	SUBI	28	28	0	32	0	
☐	4		STUR	29	28	0	0	0	
☐	5		ADDI	29	28	0	24	0	
☐	6		STUR	30	29	0	-16	0	
☐	7		STUR	0	29	0	0	0	
☐	8		SUBIS	31	0	0	1	0	
☐	9		B.LE	0	0	0	0	0	L1
☐	10		SUBI	0	0	0	1	0	
☒	11		BL	0	0	0	0	0	FACT
☐	12		LDUR	0	29	0	0	0	
☐	13		MUL	1	0	1	0	0	
☐	14		B	0	0	0	0	0	DONE
☐	15	L1	ADDI	1	31	0	1	0	
☐	16	DONE	LDUR	30	29	0	-16	0	
☐	17		LDUR	29	29	0	-24	0	
☐	18		ADDI	28	28	0	32	0	
☐	19		BR	30	0	0	0	0	

Clear Breakpoint and Run

> Set Breakpoint @ Line 11

> Run

> Clear Breakpoint @ Line 11

> Run

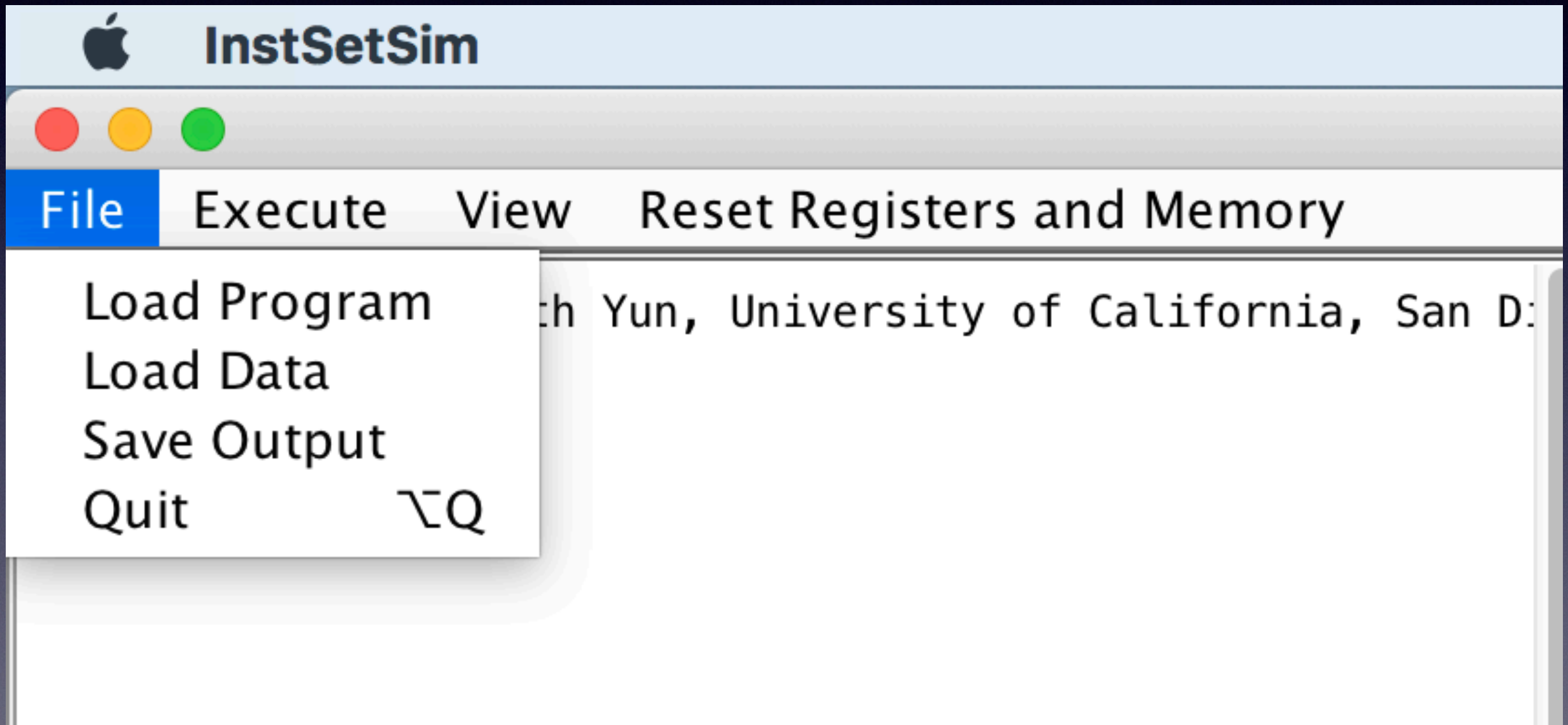
>

Program										
BP	Line	Label	Opcode	Rd/Rt	Rn	Rm	Imm/Offset	Shamt	Target	
☐	0		ADDI	0	31	0	10	0		
☐	1		BL	0	0	0	0	0	FACT	
☒	2		STOP	0	0	0	0	0		
☐	3	FACT	SUBI	28	28	0	32	0		
☐	4		STUR	29	28	0	0	0		
☐	5		ADDI	29	28	0	24	0		
☐	6		STUR	30	29	0	-16	0		
☐	7		STUR	0	29	0	0	0		
☐	8		SUBIS	31	0	0	1	0		
☐	9		B.LE	0	0	0	0	0	L1	
☐	10		SUBI	0	0	0	1	0		
☐	11		BL	0	0	0	0	0	FACT	
☐	12		LDUR	0	29	0	0	0		
☐	13		MUL	1	0	1	0	0		
☐	14		B	0	0	0	0	0	DONE	
☐	15	L1	ADDI	1	31	0	1	0		
☐	16	DONE	LDUR	30	29	0	-16	0		
☐	17		LDUR	29	29	0	-24	0		
☐	18		ADDI	28	28	0	32	0		
☐	19		BR	30	0	0	0	0		

Registers and Data Memory

[illegible]

Load Data



Assembler Directives

- Used for loading data into data memory
 - .ascii : ASCII string terminated by '\0'
 - .long : long integer
 - .float : IEEE 754 single precision
 - .double : IEEE 754 double precision

AD Example

.long

1 2 3 65536 0x7fffffffffffffffff

.float

pi: 3.14, 3.14159

e: 2.78

.double

0x4008000000000000, -0.3333333333

.ascii

bb: "Bumble bee"

"hello, world\n", "tab\ttab\ttab"

Automatic Data Alignment

- `long/double` to the nearest double word (8 byte) boundary
- `float` to the nearest word (4 byte) boundary
- `asciiz` adds `'\0'` to the end of the string

Back to AD Example

Copyright (c) Kenneth Yun, Un

> Load Data example-data

```
1
2
3
65536
9223372036854775807
3.14
3.14159
2.78
-3.0
-0.333333333
"Bumble bee"
"hello, world\n"
"tab\ttab\ttab"
```

PI: 00028

E: 00030

BB: 00048

Address	+0/+00	+1/+08	+2/+10	+3/+18	+4/+20	+5/+28
Registers						
X[00]	0000000000000000	0000000000000000	0000000000000000	0000000000000000	0000000000000000	0000000000000000
X[08]	0000000000000000	0000000000000000	0000000000000000	0000000000000000	0000000000000000	0000000000000000
X[16]	0000000000000000	0000000000000000	0000000000000000	0000000000000000	0000000000000000	0000000000000000
X[24]	0000000000000000	0000000000000000	0000000000000000	0000000000000000	0000000000000000	0000000000000000
PC	0000000000000000	NZVC	0000		SP	F
F[00]	0000000000000000	0000000000000000	0000000000000000	0000000000000000	0000000000000000	0000000000000000
F[08]	0000000000000000	0000000000000000	0000000000000000	0000000000000000	0000000000000000	0000000000000000
F[16]	0000000000000000	0000000000000000	0000000000000000	0000000000000000	0000000000000000	0000000000000000
F[24]	0000000000000000	0000000000000000	0000000000000000	0000000000000000	0000000000000000	0000000000000000
Data						
D[0000]	0000000000000001	0000000000000002	0000000000000003	0000000000001000	7FFFFFFFFFFFFFFF	4
D[00040]	BFD5555554F9B516	6220656C626D7542	6F6C6C6568006565	0A646C726F77202C	6261740962617400	0
D[00080]	0000000000000000	0000000000000000	0000000000000000	0000000000000000	0000000000000000	0
D[000C0]	0000000000000000	0000000000000000	0000000000000000	0000000000000000	0000000000000000	0
D[00100]	0000000000000000	0000000000000000	0000000000000000	0000000000000000	0000000000000000	0
D[00140]	0000000000000000	0000000000000000	0000000000000000	0000000000000000	0000000000000000	0
D[00180]	0000000000000000	0000000000000000	0000000000000000	0000000000000000	0000000000000000	0
D[001C0]	0000000000000000	0000000000000000	0000000000000000	0000000000000000	0000000000000000	0

Back to AD Example

[illegible]

pseudo-instructions

- `CMP X1, X2` = `SUBS XZR, X1, X2`
- `CMPI X1, #3` = `SUBIS XZR, X1, #3`
- `MOV X1, X2` = `ADD X1, X2, XZR`

LDA (Load Address)

- Used for loading memory address of data corresponding to label

- Recall

bb: "bumble bee"

- LDA X1, bb $\Rightarrow$ X1 = 0x48

```
> Load Data example-data
```

```
1
2
3
65536
9223372036854775807
3.14
3.14159
2.78
-3.0
-0.333333333
"Bumble bee"
"hello, world\n"
"tab\ttab\ttab"
```

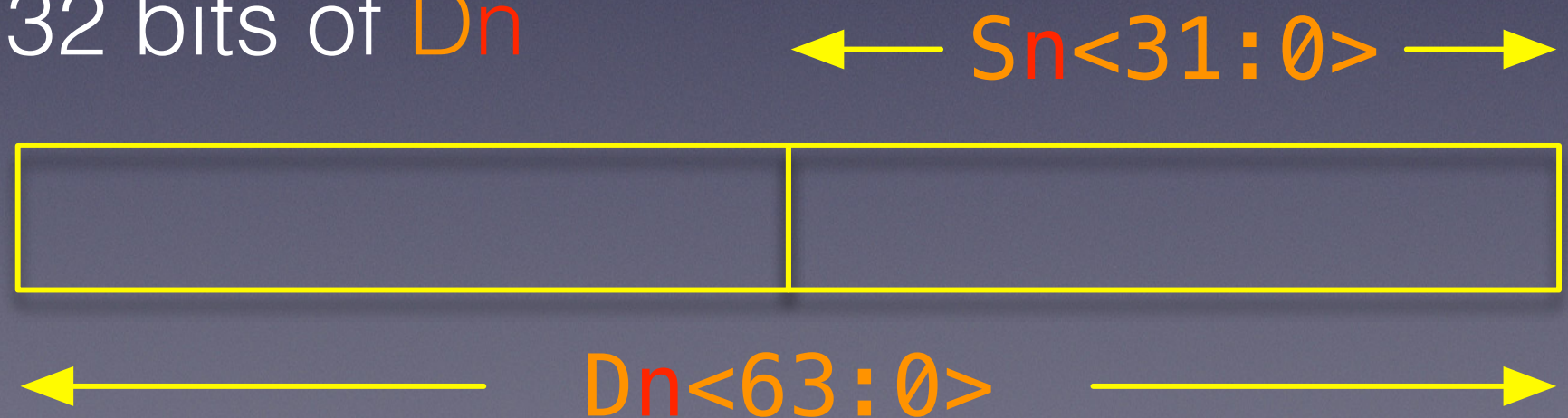
```
PI:      00028
```

```
E:       00030
```

```
BB:      00048
```

Floating-Point

- *legiss* supports both single and double precision floating-point arithmetic
- Single/double precision registers: S0–S31 / D0–D31
- Single precision register S_n uses the least significant 32 bits of D_n



Floating-Point Registers

PC ▶	0	NZVC ▶
F[00]	000000000000000000000000	00000000
F[08]	000000000000000000000000	00000000
F[16]	000000000000000000000000	00000000
F[24]	000000000000000000000000	00000000
Data		

D24 (circled in red) is the 24-bit data field in the F[24] register.

S8 (circled in red) is the 8-bit sign field in the F[08] register.

LEGv8 FP Instructions

Single Precision

FADDS

FSUBS

FMULS

FDIVS

FCMPS

LDURS

STURS

Double Precision

FADDD

FSUBD

FMULD

FDIVD

FCMPD

LDURD

STURD

Loading/Storing FP Numbers

```
LDURD D1, [X2, #offset]  
    // D1 ← Memory[X2 + #offset]
```

```
STURD D1, [X2, #offset]  
    // Memory[X2 + #offset] ← D1
```